

Coding and Solution

Implementation approach, feature flow, and delivery plan

Project planning document • Phase 5: Project Development

Purpose

This document turns the project development phase into a practical, reviewable guide for the Comic Craft application. It is written for student developers and project reviewers.

1. Project problem and proposed solution

Comic Craft is a digital space for discovering, organizing, and reading comics. The application should make the main journey clear: find a comic, inspect its details, and read or save it. The development phase translates that journey into small features that can be built and reviewed.

Need	Solution direction
Discover	Browse a catalog and search or filter comics.
Understand	Show cover, title, creator, description, and reading status.
Read	Provide a focused reader view with clear navigation.
Keep track	Allow a user to save favorites or maintain a reading list.

2. Core user flow

Reader journey

- Open the catalog and browse available comics.
- Search or choose a category to narrow results.
- Open a comic detail page and review its information.
- Start reading, save it, or return to the catalog.
- Show helpful states if content is loading, unavailable, or not found.

Build the smallest complete journey first: catalog → details → reader. Add optional enhancements after that path works end to end.

3. Implementation stages

Stage	Work and output
1. Foundation	Set up the application, routes, shared styles, and sample data.
2. Catalog	Create comic cards, search/filter behavior, and empty states.
3. Details	Add comic information and actions such as read or save.
4. Reader	Implement page navigation and a readable viewing layout.
5. Persistence	Store favorites or progress using the selected storage approach.
6. Polish	Check responsive layout, accessibility, errors, and project documentation.

4. Data and interface boundaries

Keep comic information in a consistent model so catalog, detail, and reader views use the same source. A practical starter record can include an id, title, creator, description, cover image, category, and page list.

Separate data access from display so the project can replace sample data with an API later.

- Validate required fields before displaying a comic.
- Use stable identifiers rather than array positions for saved items.
- Show a clear fallback when a cover or page image cannot load.
- Keep user-specific actions behind a service or storage boundary.

5. Risks and responses

Risk	Response
Scope grows too quickly	Prioritize the core browse-to-read journey and defer extras.
Image assets are missing	Use approved sample assets and graceful placeholders.
Different code styles	Agree conventions early and review changes consistently.
Data source changes	Use a small service layer and a stable comic model.
Reading view is hard to use	Review contrast, controls, keyboard access, and small screens.

6. Delivery checklist

- Catalog and detail views use the same comic records.
- Search and filters have clear results and no-results behavior.
- Reader navigation is understandable and works on the target screen sizes.
- Loading, empty, missing-image, and error states are covered.
- Setup and feature notes are available to the next contributor.

The coding solution is ready for handoff when the core user flow is complete, known limitations are written down, and the team can explain how each feature connects to the shared data model.